

KIỂM THỬ PHẦN MỀM

TÀI LIỆU THỰC HÀNH

LAB 9 — PAGE OBJECT MODEL NÂNG CAO + DATA-DRIVEN TESTING

Nội dung	Chi tiết
Chủ đề	Page Object Model (POM), BasePage, BaseTest, Data-Driven Testing (DDT)
Kỹ năng đạt được	Xây dựng framework test chuẩn, tách dữ liệu khỏi code
Công cụ	Selenium 4.x, TestNG 7.x, Apache POI, Jackson, Java Faker, Maven
Thời gian	4 tiết Điểm số: 10 điểm
Điều kiện tiên quyết	Đã thực hiện xong các lab trước (Selenium WebDriver cơ bản, TestNG Suites)
Website thực hành	saucedemo.com, demoqa.com

Mục tiêu bài lab

- Sau khi hoàn thành Lab 9, sinh viên có thể:
- Giải thích tại sao cần Page Object Model và khi nào nên áp dụng
 - Xây dựng BasePage và BaseTest tái sử dụng cho toàn dự án
 - Thiết kế cấu trúc thư mục project chuẩn
 - Đọc test data từ file Excel (.xlsx) và JSON bằng code Java
 - Triển khai ConfigReader để framework chạy được trên nhiều môi trường (dev/staging/prod)
 - Viết RetryAnalyzer để tự động chạy lại test bị fail do lỗi mạng/flakiness
 - Tích hợp @DataProvider với nguồn dữ liệu bên ngoài

I. TÓM TẮT LÝ THUYẾT

Trang web tham khảo:

- <https://www.automationtestinginsider.com/p/selenium-vi.html>
- <https://blog.ntechdevelopers.com/test-automation-approach-phuong-phap-kiem-thu-tu-dong/>
- [OBJECT](#)
- <https://200lab.io/blog/autmation-testing-la-gi>
- <https://centic9.github.io/github-version-statistics/resultsCurrent.html?type=all>

9.1 Vấn đề với cách viết test (Naive Test).

Khi mới học Selenium , <https://www.selenium.dev/>, hầu hết mọi người viết test theo kiểu nhúng tất cả vào một class: locator, logic nghiệp vụ và dữ liệu test đều lẫn lộn. Cách này hoạt động tốt khi mới bắt đầu, nhưng gây ra 4 vấn đề nghiêm trọng trong dự án thực tế có nhiều tính năng và nhiều người cùng làm:

Vấn đề	Biểu hiện cụ thể	Hậu quả trong dự án thực
Code trùng lặp (Duplication)	By.id("username") xuất hiện ở 10 class khác nhau	Khi UI thay đổi id → sửa 10 chỗ, dễ bỏ sót, mất nhiều giờ
Kết dính chặt (Tight coupling)	Logic điều hướng trộn lẫn với assertion trong cùng method	Không tái sử dụng được step, khó đọc, khó debug
Dữ liệu cứng (Hardcoded data)	username="standard user" nằm trực tiếp trong code	Không chạy được ở môi trường staging/production khác
Không thể mở rộng	Thêm 1 test case mới = copy-paste toàn bộ method	Team 5 người cùng sửa 1 file → conflict git liên tục

So sánh trực tiếp Naive Test vs POM Test:

```
// NAIVE TEST – Antipattern, tuyệt đối tránh trong dự án thực
@Test
public void testLoginSuccess_WRONG() {
    WebDriverManager.chromedriver().setup();
    WebDriver driver = new ChromeDriver();
    driver.get("https://www.saucedemo.com");
    driver.findElement(By.id("user-name")).sendKeys("standard_user"); // hardcoded
    locator
    driver.findElement(By.id("password")).sendKeys("secret_sauce"); // hardcoded data
    driver.findElement(By.id("login-button")).click();
    Assert.assertTrue(driver.getCurrentUrl().contains("inventory"));
    driver.quit();
}
```

```
// POM TEST – Chuẩn frame
@Test
public void testLoginSuccess_CORRECT() {
    LoginPage loginPage = new LoginPage(getDriver()); // Page Object
    loginPage.login(config.getUsername(), config.getPassword()); // data từ config
    Assert.assertTrue(new InventoryPage(getDriver()).isLoading(), "Trang inventory chưa load");
}
// Khi id="user-name" đổi thành id="username" → chỉ sửa LoginPage.java, không đụng test class
```

9.2 Page Object Model - Nguyên tắc và Kiến trúc POM

Tham khảo thêm các tài liệu:

- <https://anhtester.com/lesson/playwright-typescript-bai-10-page-object-model-pom-nguyen-tac-va-cau-truc-nen-tang-trong-playwright>
- <https://viblo.asia/p/page-object-model-pom-trong-selenium-bWrZn7MwIw>
- <https://vananhthooo.wordpress.com/2017/09/12/gioi-thieu-ve-page-object-model-framework/>
- <https://viettuts.vn/selenium/page-object-model-trong-selenium>

🚩 Nguyên tắc cốt lõi của POM

Nguyên tắc 1: Mỗi trang web là một class Java:

Mỗi trang (LoginPage, InventoryPage, CartPage...) được đại diện bởi một class riêng.

Class đó chứa: locator của phần tử + action method tương tác với trang.

Nguyên tắc 2: Test class KHÔNG findElement trực tiếp:

Test class chỉ gọi method của Page Object.

Ví dụ: loginPage.login(user, pass) thay vì
driver.findElement(By.id("u")).sendKeys(user)

Nguyên tắc 3: Method trả về Page Object (Fluent Interface):

Sau khi click "Login" thành công → trả về InventoryPage (trang tiếp theo).

Cho phép viết chuỗi:

loginPage.login(u,p).addFirstItemToCart().goToCart().checkout()

Nguyên tắc 4: Assertion KHÔNG nằm trong Page Object:

Page Object chỉ chứa hành động (action), không kiểm tra đúng/sai.

Mọi Assert.assertEquals / Assert.assertTrue nằm trong test class.

Rút ra được Cấu trúc thư mục project POM chuẩn:

```

SeleniumFramework/
├── src/
│   ├── main/java/framework/
│   │   ├── base/
│   │   │   ├── BasePage.java      ← Lớp cha: waitAndClick, waitAndType, getText...
│   │   │   └── BaseTest.java      ← Lớp cha: setup driver, teardown, screenshot
│   │   ├── config/
│   │   │   └── ConfigReader.java  ← Đọc config.properties theo môi trường
│   │   ├── pages/
│   │   │   ├── LoginPage.java     ← Page Object cho trang đăng nhập
│   │   │   ├── InventoryPage.java ← Page Object cho trang sản phẩm
│   │   │   └── CartPage.java       ← Page Object cho trang giỏ hàng
│   │   └── utils/
│   │       ├── ExcelReader.java   ← Đọc test data từ file Excel .xlsx
│   │       ├── JsonReader.java    ← Đọc test data từ file JSON
│   │       ├── ScreenshotUtil.java ← Chụp ảnh màn hình khi test fail
│   │       └── RetryAnalyzer.java  ← Tự động chạy lại test bị fail
│   └── test/
│       ├── java/tests/
│       │   ├── LoginTest.java
│       │   ├── LoginDDTTest.java  ← Test dùng dữ liệu từ Excel/JSON
│       │   └── CheckoutTest.java
│       └── resources/
│           ├── config-dev.properties
│           ├── config-staging.properties
│           ├── testdata/
│           │   ├── login_data.xlsx ← Dữ liệu test đăng nhập
│           │   └── users.json      ← Dữ liệu user dạng JSON
│           ├── testng-smoke.xml
│           └── testng-regression.xml
└── pom.xml

```

Thêm các dependency cần thiết vào pom.xml:

```

<dependencies>
  <!-- Selenium WebDriver -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.18.1</version>
  </dependency>
  <!-- WebDriverManager -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.8.0</version>
  </dependency>
  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.9.0</version>
    <scope>test</scope>
  </dependency>
  <!-- Apache POI: Đọc file Excel -->
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>

```

```

    <version>5.2.5</version>
  </dependency>
  <!-- Jackson: Đọc/ghi JSON -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
  </dependency>
  <!-- Java Faker: Sinh dữ liệu test ngẫu nhiên -->
  <dependency>
    <groupId>com.github.javafaker</groupId>
    <artifactId>javafaker</artifactId>
    <version>1.0.2</version>
  </dependency>
</dependencies>

```

9.3 BasePage - Lớp nền tảng cho mọi Page Object

BasePage đóng gói tất cả các thao tác WebDriver phổ biến thành các method có Explicit Wait tích hợp sẵn. Mọi Page Object đều extends BasePage để dùng lại các method này:

```

// BasePage.java
package framework.base;

public abstract class BasePage {
    protected WebDriver driver;
    protected WebDriverWait wait;

    public BasePage(WebDriver driver) {
        this.driver = driver;
        // Explicit wait 15 giây – đủ cho mạng chậm, không chờ lãng phí
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        PageFactory.initElements(driver, this); // Khởi tạo @FindBy annotations
    }

    // Chờ element clickable rồi mới click – tránh ElementNotInteractableException
    protected void waitAndClick(WebElement element) {
        wait.until(ExpectedConditions.elementToBeClickable(element)).click();
    }

    // Xóa nội dung cũ rồi gõ – tránh dữ liệu bị nối thêm
    protected void waitAndType(WebElement element, String text) {
        wait.until(ExpectedConditions.visibilityOf(element));
        element.clear();
        element.sendKeys(text);
    }

    // Lấy text đã trim whitespace
    protected String getText(WebElement element) {
        return wait.until(ExpectedConditions.visibilityOf(element)).getText().trim();
    }

    // Kiểm tra hiển thị – KHÔNG throw exception nếu không tìm thấy
    protected boolean isElementVisible(By locator) {
        try {
            return driver.findElement(locator).isDisplayed();
        } catch (NoSuchElementException | StaleElementReferenceException e) {

```

```

        return false;
    }
}

// Cuộn trang đến element – xử lý element nằm ngoài viewport
protected void scrollToElement(WebElement element) {
    ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);",
element);
}

// Chờ trang load xong – dùng sau khi điều hướng
protected void waitForPageLoad() {
    wait.until(driver -> ((JavascriptExecutor) driver)
        .executeScript("return document.readyState").equals("complete"));
}

// Lấy giá trị attribute
protected String getAttribute(WebElement element, String attr) {
    return wait.until(ExpectedConditions.visibilityOf(element)).getAttribute(attr);
}
}

```

9.4 Ví dụ Page Object thực tế - LoginPage và InventoryPage

```

// LoginPage.java
package framework.pages;

public class LoginPage extends BasePage {

    // @FindBy thay thế driver.findElement() – định nghĩa locator một lần duy nhất
    @FindBy(id = "user-name")
    private WebElement usernameField;

    @FindBy(id = "password")
    private WebElement passwordField;

    @FindBy(id = "login-button")
    private WebElement loginButton;

    @FindBy(css = "[data-test='error']")
    private WebElement errorMessage;

    public LoginPage(WebDriver driver) { super(driver); }

    // Method trả về InventoryPage – Fluent Interface
    public InventoryPage login(String username, String password) {
        waitAndType(usernameField, username);
        waitAndType(passwordField, password);
        waitAndClick(loginButton);
        return new InventoryPage(driver);
    }

    // Đăng nhập khi biết trước sẽ thất bại – trả về chính LoginPage
    public LoginPage loginExpectingFailure(String username, String password) {
        waitAndType(usernameField, username);
        waitAndType(passwordField, password);
    }
}

```

```
        waitAndClick(loginButton);
        return this;
    }

    public String getErrorMessage() {
        return getText(errorMessage);
    }

    public boolean isErrorDisplayed() {
        return isElementVisible(By.cssSelector("[data-test='error']"));
    }
}

// InventoryPage.java
public class InventoryPage extends BasePage {

    @FindBy(css = ".inventory_list")
    private WebElement inventoryList;

    @FindBy(css = ".shopping_cart_badge")
    private WebElement cartBadge;

    @FindBy(css = ".inventory_item button")
    private List<WebElement> addToCartButtons;

    public InventoryPage(WebDriver driver) { super(driver); }

    // Kiểm tra trang đã load thành công
    public boolean isLoaded() {
        return isElementVisible(By.cssSelector(".inventory_list"));
    }

    // Thêm sản phẩm đầu tiên vào giỏ
    public InventoryPage addFirstItemToCart() {
        waitAndClick(addToCartButtons.get(0));
        return this;
    }

    // Lấy số lượng item trong badge giỏ hàng
    public int getCartItemCount() {
        try {
            return Integer.parseInt(cartBadge.getText().trim());
        } catch (Exception e) {
            return 0; // Badge không hiển thị = giỏ hàng rỗng
        }
    }

    public CartPage goToCart() {
        driver.findElement(By.className("shopping_cart_link")).click();
        return new CartPage(driver);
    }
}
```


9.5 BaseTest - Quản lý vòng đời WebDriver

BaseTest đảm nhận việc tạo và dọn dẹp WebDriver cho mỗi test method. Dùng ThreadLocal để hỗ trợ chạy song song an toàn (đã học và làm ở Lab trước):

```
// BaseTest.java
package framework.base;

public abstract class BaseTest {
    private static ThreadLocal<WebDriver> tlDriver = new ThreadLocal<>();

    protected WebDriver getDriver() { return tlDriver.get(); }

    @Parameters({"browser", "env"})
    @BeforeMethod(alwaysRun = true)
    public void setUp(@Optional("chrome") String browser,
                     @Optional("dev") String env) {
        // Đặt env làm System property để ConfigReader đọc đúng file
        System.setProperty("env", env);

        WebDriver driver = DriverFactory.createDriver(browser);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
        driver.get(ConfigReader.getInstance().getBaseUrl());
        tlDriver.set(driver);
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult result) {
        // Chụp ảnh TRƯỚC khi quit - bắt buộc trong dự án thực để debug
        if (result.getStatus() == ITestResult.FAILURE) {
            String screenshotPath = ScreenshotUtil.capture(getDriver(),
result.getName());
            // Đính kèm vào Allure report nếu đang dùng Allure
            Allure.addAttachment("Screenshot khi fail",
                new ByteArrayInputStream(ScreenshotUtil.captureAsBytes(getDriver())));
        }
        if (getDriver() != null) {
            getDriver().quit();
            tlDriver.remove(); // Quan trọng: tránh memory leak khi chạy song song
        }
    }
}
```

9.6 Data-Driven Testing (DDT) - Tách dữ liệu khỏi code

Tham khảo:

- <https://viblo.asia/p/data-driven-testing-framework-6J3Zg9Ng5mB>
- <https://haibgit.blogspot.com/2018/10/data-driven-testing-in-robot-framework.html>
- <https://anhtester.com/lesson/cucumber-testng-bai-1-gioi-thieu-huong-phat-trien-tdd-va-bdd-trong-code-automation-test>

❖ **Tại sao phải tách dữ liệu test ra khỏi code Java?**

Trong dự án thực tế, QA Lead quản lý hàng trăm test case. Người viết test data thường không phải là người viết code — đó có thể là QA không biết lập trình, hoặc Business Analyst cung cấp dữ liệu nghiệp vụ.

Nếu dữ liệu nằm trong code Java:

- Mỗi lần thêm/sửa test case phải build lại project (mất 2-5 phút)
- Người không biết Java không cập nhật được
- Dữ liệu production (mật khẩu thật) có thể lọt vào git repository

Nếu dữ liệu nằm trong file Excel/JSON:

- QA mở Excel, thêm dòng mới → chạy lại test ngay, không cần build
- Dữ liệu nhạy cảm không commit vào git (thêm vào .gitignore)
- Dễ audit: ai thêm/sửa test case khi nào

Nguồn 1: @DataProvider thuần Java — Dùng cho dữ liệu nhỏ, cố định:

```
// Cách đơn giản nhất — không cần file ngoài
@DataProvider(name = "loginData", parallel = true) // parallel=true: chạy song song
public Object[][] provideLoginData() {
    return new Object[][] {
        // {username, password, expectedResult, testDescription}
        {"standard_user", "secret_sauce", "SUCCESS", "Đăng nhập hợp lệ"},
        {"locked_out_user", "secret_sauce", "Sorry, this user has been locked out",
"User bị khoá"},
        {"problem_user", "secret_sauce", "SUCCESS", "User có vấn đề giao diện"},
        {"", "secret_sauce", "Username is required", "Username rỗng"},
        {"standard_user", "", "Password is required", "Password rỗng"},
        {"standard_user", "wrongpass", "Username and password do not match", "Sai
mật khẩu"},
        {"<script>alert(1)", "secret_sauce", "Username and password do not match", "XSS
injection"},
    };
}
```

Nguồn 2: Đọc từ file Excel (.xlsx) — Dùng khi QA không biết code:

```
// ExcelReader.java
public class ExcelReader {

    /**
     * Đọc toàn bộ data từ một sheet Excel
     * Dòng đầu tiên (index 0) được coi là header — bị bỏ qua
     * Trả về Object[][] để dùng với @DataProvider
     */
}
```

```

public static Object[][] getData(String filePath, String sheetName) {
    try (FileInputStream fis = new FileInputStream(filePath);
        Workbook workbook = new XSSFWorkbook(fis)) {

        Sheet sheet = workbook.getSheet(sheetName);
        if (sheet == null)
            throw new RuntimeException("Không tìm thấy sheet: " + sheetName);

        int lastRow = sheet.getLastRowNum();
        int lastCol = sheet.getRow(0).getLastCellNum();
        Object[][] data = new Object[lastRow][lastCol];

        for (int r = 1; r <= lastRow; r++) { // Bắt đầu từ dòng 1: bỏ header
            Row row = sheet.getRow(r);
            if (row == null) continue;
            for (int c = 0; c < lastCol; c++) {
                Cell cell = row.getCell(c);
                data[r-1][c] = getCellValue(cell);
            }
        }
        return data;

    } catch (IOException e) {
        throw new RuntimeException("Lỗi đọc Excel: " + filePath, e);
    }
}

// Xử lý từng kiểu dữ liệu trong cell
private static String getCellValue(Cell cell) {
    if (cell == null) return ""; // Cell rỗng → chuỗi rỗng, không
    NullPointerException
    return switch (cell.getCellType()) {
        case STRING -> cell.getStringCellValue().trim();
        case NUMERIC -> String.valueOf((long) cell.getNumericCellValue());
        case BOOLEAN -> String.valueOf(cell.getBooleanCellValue());
        case FORMULA -> cell.getCachedFormulaResultType() == CellType.NUMERIC
            ? String.valueOf((long) cell.getNumericCellValue())
            : cell.getStringCellValue();
        default -> "";
    };
}

// Sử dụng trong test class:
@DataProvider(name = "excelLoginData")
public Object[][] getLoginDataFromExcel() {
    String path = "src/test/resources/testdata/login_data.xlsx";
    return ExcelReader.getData(path, "LoginCases");
}

@Test(dataProvider = "excelLoginData")
public void testLoginFromExcel(String username, String password, String expected) {
    LoginPage login = new LoginPage(getDriver());
    // logic test...
}

```

Nguồn 3: Đọc từ file JSON — Dùng cho dữ liệu có cấu trúc phức tạp:

```
// users.json — Dữ liệu dạng mảng object
[
  { "username": "standard_user",    "password": "secret_sauce",  "role": "user",
    "expectSuccess": true },
  { "username": "locked_out_user",  "password": "secret_sauce",  "role": "user",
    "expectSuccess": false },
  { "username": "admin_user",       "password": "admin_pass",    "role": "admin",
    "expectSuccess": true  }
]

// UserData.java — POJO mapping
public class UserData {
    public String username;
    public String password;
    public String role;
    public boolean expectSuccess;
}

// JsonReader.java
public class JsonReader {
    public static List<UserData> readUsers(String filePath) throws IOException {
        ObjectMapper mapper = new ObjectMapper();
        return mapper.readValue(new File(filePath),
                                new TypeReference<List<UserData>>() {});
    }
}

// Trong test class:
@DataProvider(name = "jsonUsers")
public Object[][] getUsersFromJson() throws IOException {
    List<UserData> users =
        JsonReader.readUsers("src/test/resources/testdata/users.json");
    return users.stream()
        .map(u -> new Object[]{u.username, u.password, u.expectSuccess})
        .toArray(Object[][]::new);
}
```

9.7 ConfigReader - Chạy test trên nhiều môi trường

- <https://cafeincode.com/loi-tren-moi-truong-staging-ma-khong-ai-chiu-sua-den-cung/>

Trong dự án thực, cùng bộ test phải chạy được ở dev, staging và production.

ConfigReader đọc đúng file cấu hình theo biến môi trường truyền vào khi chạy:

Trong dự án thực, cùng bộ test phải chạy được ở dev, staging và production.

ConfigReader đọc đúng file cấu hình theo biến môi trường truyền vào khi chạy:

```
# config-dev.properties
base.url=https://www.saucedemo.com
browser=chrome
explicit.wait=15
implicit.wait=5
screenshot.path=target/screenshots/
retry.count=1
```

```
# config-staging.properties
base.url=https://staging.saucedemo.com
browser=chrome
explicit.wait=20
implicit.wait=8
screenshot.path=target/screenshots/
retry.count=2

// ConfigReader.java – Singleton Pattern
public class ConfigReader {
    private static final Properties props = new Properties();
    private static ConfigReader instance;

    private ConfigReader() {
        String env = System.getProperty("env", "dev");
        String file = "src/test/resources/config-" + env + ".properties";
        try (FileInputStream fis = new FileInputStream(file)) {
            props.load(fis);
            System.out.println("[ConfigReader] Đang dùng môi trường: " + env);
        } catch (IOException e) {
            throw new RuntimeException("Không tìm thấy config: " + file);
        }
    }

    public static ConfigReader getInstance() {
        if (instance == null) instance = new ConfigReader();
        return instance;
    }

    public String getBaseUrl() { return props.getProperty("base.url"); }
    public String getBrowser() { return props.getProperty("browser", "chrome"); }
    public int getExplicitWait() { return
Integer.parseInt(props.getProperty("explicit.wait", "15")); }
    public int getRetryCount() { return
Integer.parseInt(props.getProperty("retry.count", "1")); }
    public String getScreenshotPath() { return props.getProperty("screenshot.path"); }
}

// Chạy ở môi trường staging từ command line:
// mvn test -Denv=staging
// Chạy ở môi trường dev (mặc định):
// mvn test
```

9.8 RetryAnalyzer - Tự động chạy lại test bị fail

Trong dự án thực, một số test bị fail không phải do lỗi code mà do mạng chậm, server lag tạm thời, hay race condition giao diện. RetryAnalyzer giúp phân biệt “lỗi thật” với “lỗi thoáng qua”:

<https://gpcoder.com/5384-lam-the-nao-de-chay-lai-mot-failed-test-trong-junit/>

```
// RetryAnalyzer.java
public class RetryAnalyzer implements IRetryAnalyzer {
    private int retryCount = 0;

    @Override
    public boolean retry(ITestResult result) {
        int maxRetry = ConfigReader.getInstance().getRetryCount();
        if (retryCount < maxRetry) {
            retryCount++;
            System.out.println("[Retry] Đang chạy lại lần " + retryCount
                               + " cho test: " + result.getName());
            return true;    // true = chạy lại
        }
        return false;      // false = dùng, đánh dấu FAIL
    }
}

// RetryListener.java – Áp dụng RetryAnalyzer cho mọi test tự động
public class RetryListener implements IAnnotationTransformer {
    @Override
    public void transform(ITestAnnotation annotation, Class testClass,
                          Constructor testConstructor, Method testMethod) {
        annotation.setRetryAnalyzer(RetryAnalyzer.class);
        // Không cần thêm retryAnalyzer = RetryAnalyzer.class vào từng @Test
    }
}

// testng.xml – Đăng ký listener
// <listeners>
//   <listener class-name="framework.utils.RetryListener"/>
// </listeners>
```

II. BÀI TẬP THỰC HÀNH

Bài 1 XÂY DỰNG BASEPAGE VÀ BASETEST (1.0 điểm)

Yêu cầu: Tạo lớp nền tảng cho toàn bộ framework. Đây là bước quan trọng nhất - mọi code sau đều phụ thuộc vào 2 lớp này.

BasePage (0.5 điểm):

- Triển khai đầy đủ 7 method: `waitAndClick`, `waitAndType`, `getText`, `isElementVisible`, `scrollToElement`, `waitForPageLoad`, `getAttribute`
- Tất cả method có Explicit Wait tích hợp - KHÔNG dùng `Thread.sleep()`
- Xử lý `StaleElementReferenceException` trong `isElementVisible` - thực tế hay gặp khi trang render lại DOM
- Viết Javadoc cho từng method giải thích mục đích và trường hợp dùng

BaseTest (0.5 điểm):

- `@BeforeMethod` nhận 2 tham số từ `testng.xml`: `browser` và `env`. Dùng `@Optional` để có giá trị mặc định
- `@AfterMethod` chụp ảnh màn hình và lưu vào thư mục `target/screenshots/` với tên = `{testName}_{timestamp}.png`
- Dùng `ThreadLocal<WebDriver>` - KHÔNG dùng biến static
- Kiểm tra: Chạy song song `parallel="methods" thread-count="3"` → 3 cửa sổ Chrome mở cùng lúc, không lỗi

Tiêu chí chấm điểm

- BasePage: comment out 1 method bất kỳ → compilation error ngay (chứng tỏ page objects phụ thuộc vào BasePage)
- BaseTest: Chạy test fail có ý thức (sai assertion) → ảnh chụp màn hình xuất hiện trong `target/screenshots/`
- Không dùng `Thread.sleep()` bất kỳ chỗ nào trong BasePage (dùng grep để kiểm tra)

Bài 2 XÂY DỰNG PAGE OBJECTS CHO SAUCEDEMO (1.0 điểm)

Tạo 3 Page Object cho website saucedemo.com. Áp dụng đúng 4 nguyên tắc POM tìm hiểu thêm phần lý thuyết ở các link đã đưa bên trên.

Page Object	Phần tử cần định nghĩa (@FindBy)	Action method cần triển khai
LoginPage	username field, password field, login button, error message	login(user,pass) → InventoryPage loginExpectingFailure(user,pass) → LoginPage getErrorMessage() → String isErrorDisplayed() → boolean
InventoryPage	inventory list, cart badge, danh sách sản phẩm, nút Add to Cart	isLoading() → boolean addFirstItemToCart() → InventoryPage addItemByName(name) → InventoryPage getCartItemCount() → int goToCart() → CartPage
CartPage	danh sách item trong giỏ, nút Remove, nút Checkout	getItemCount() → int removeFirstItem() → CartPage goToCheckout() → CheckoutPage getItemNames() → List<String>

Yêu cầu kỹ thuật:

- Test class KHÔNG được chứa driver.findElement() hay By.id() - vi phạm sẽ bị trừ điểm
- Fluent Interface: loginPage.login(u,p).addFirstItemToCart().goToCart() phải hoạt động
- Xử lý trường hợp CartPage không có item - getItemCount() trả về 0, không throw exception
- Viết LoginTest và CartTest kế thừa BaseTest - tổng cộng ít nhất 6 test method

Bài 3 DATA-DRIVEN TESTING VỚI EXCEL (1.0 điểm)

Xây dựng bộ kiểm thử đăng nhập data-driven. Dữ liệu lưu trong file Excel

login_data.xlsx với 3 sheet:

Sheet	Mục đích	Số dòng data tối thiểu	Các cột bắt buộc
SmokeCases	Happy path — đăng nhập thành công	3 dòng	username password expected_url description
NegativeCases	Đăng nhập thất bại: sai pass, khoá tài khoản, rỗng	5 dòng	username password expected_error description
BoundaryCases	Dữ liệu biên: chuỗi rất dài, ký tự đặc biệt, SQL pattern	4 dòng	username password expected_error description

Yêu cầu kỹ thuật:

- ExcelReader.getCellValue() phải xử lý cell null và 4 kiểu dữ liệu: STRING, NUMERIC, BOOLEAN, FORMULA
- Khi thêm dòng mới vào Excel → test tự động chạy thêm case đó, không cần sửa code Java
- Tên test method trong TestNG Report phải hiển thị description từ Excel (không chỉ là testLoginFromExcel[0])
- Cấu hình testng.xml: chạy groups "smoke" → chỉ đọc SmokeCases sheet; groups "regression" → đọc tất cả 3 sheet
- Chụp màn hình TestNG HTML Report cho thấy tên test case rõ ràng từ cột description

Bài 4 DATA-DRIVEN TESTING VỚI JSON + JAVA FAKER (1.0 điểm)

Triển khai 2 tính năng DDT nâng cao dùng JSON và dữ liệu sinh ngẫu nhiên:

Phần A: JsonReader (0.5 điểm):

- Tạo file users.json chứa danh sách 5 user với các trường: username, password, role, expectSuccess, description
- Tạo class UserData POJO với Jackson @JsonProperty để mapping tên field JSON sang Java
- Triển khai JsonReader.readUsers() và dùng trong @DataProvider
- Viết UserLoginTest với 5 test case từ JSON, kiểm tra cả trường hợp SUCCESS và FAILURE

Phần B : Java Faker (0.5 điểm):

- Dùng Java Faker sinh dữ liệu ngẫu nhiên cho form checkout: firstName, lastName, postalCode
- Mỗi lần chạy test sinh dữ liệu khác nhau - chứng minh bằng cách chạy 2 lần và so sánh log
- Tạo TestDataFactory.java tập trung tất cả logic sinh data, không scatter khắp nơi trong test class

```
// TestDataFactory.java - Ví dụ gợi ý
public class TestDataFactory {
    private static final Faker faker = new Faker(new Locale("vi"));

    public static String randomFirstName() { return faker.name().firstName(); }
    public static String randomLastName() { return faker.name().lastName(); }
    public static String randomPostalCode() { return faker.number().digits(5); }
    public static String randomEmail() { return faker.internet().emailAddress(); }

    // Sinh một bộ dữ liệu checkout hoàn chỉnh
    public static Map<String,String> randomCheckoutData() {
        return Map.of(
            "firstName", randomFirstName(),
            "lastName", randomLastName(),
            "postalCode", randomPostalCode()
        );
    }
}
```

Bài 5 CONFIGREADER + ĐA MÔI TRƯỜNG (1.0 điểm)

Triển khai ConfigReader và kiểm chứng framework chạy được trên nhiều môi trường chỉ bằng 1 tham số, không sửa code:

- Tạo 2 file: config-dev.properties (URL saucedemo.com) và config-staging.properties (URL khác, explicit.wait = 20)
- ConfigReader dùng Singleton pattern, đọc đúng file theo System.getProperty("env", "dev")
- BaseTest.setUp() gọi System.setProperty("env", env) trước khi khởi tạo ConfigReader
- Demo: chạy mvn test -Denv=dev → log hiển thị "Đang dùng môi trường: dev", explicit wait = 15
- Demo: chạy mvn test -Denv=staging → log hiển thị "Đang dùng môi trường: staging", explicit wait = 20
- KHÔNG hardcode URL hay timeout ở bất kỳ đâu ngoài file config - vi phạm sẽ bị trừ điểm

Bài 6 RETRYANALYZER + FLAKY TEST SIMULATION (1 điểm)

Triển khai cơ chế retry và kiểm chứng nó hoạt động đúng bằng cách mô phỏng test không ổn định (flaky test):

- Triển khai RetryAnalyzer.java (implements IRetryAnalyzer) đọc maxRetry từ ConfigReader
- Triển khai RetryListener.java (implements IAnnotationTransformer) để áp dụng tự động cho tất cả test
- Đăng ký RetryListener vào testng.xml trong thẻ <listeners>

Tạo test mô phỏng flaky:

```
// FlakySimulationTest.java - Mô phỏng test không ổn định
public class FlakySimulationTest extends BaseTest {
    private static int callCount = 0;

    @Test(description = "Test mô phỏng flaky - fail 2 lần đầu, pass lần thứ 3")
    public void testFlakyScenario() {
        callCount++;
        System.out.println("[FlakyTest] Đang chạy lần thứ: " + callCount);
        // Mô phỏng: 2 lần đầu fail, lần thứ 3 mới pass
        if (callCount <= 2) {
            Assert.fail("Mô phỏng lỗi mạng tạm thời - lần " + callCount);
        }
        Assert.assertTrue(true, "Test pass ở lần thứ " + callCount);
    }
}
```

- Chạy test trên. TestNG Report phải cho thấy: 2 lần RETRY → 1 lần PASS cuối cùng
- Đặt retry.count=0 trong config → test fail ngay lần đầu, không retry. Chụp màn hình so sánh 2 trường hợp

Bài 7 DỰ ÁN TỔNG HỢP: REFACTOR TOÀN BỘ CODE (1.0 điểm)

Bài tập tổng hợp: Lấy toàn bộ code đã viết ở Lab 1 và Lab 2, refactor hoàn toàn sang framework POM + DDT vừa xây dựng. Đây là kỹ năng thực tế quan trọng vì trong công ty, bạn sẽ thường xuyên phải nâng cấp code cũ.

Yêu cầu:

1. Tạo git commit TRƯỚC khi refactor — để có thể so sánh before/after (dùng git diff)
2. Di chuyển tất cả locator vào Page Object tương ứng (LoginPage, InventoryPage, CartPage)
3. Xóa tất cả Thread.sleep() — thay bằng Explicit Wait trong BasePage
4. Xóa tất cả new ChromeDriver() trực tiếp — thay bằng BaseTest.getDriver()
5. Di chuyển dữ liệu test (username, password) ra file Excel hoặc @DataProvider
6. Chạy toàn bộ suite → 100% PASS. Thời gian chạy phải giảm so với code cũ (do loại Thread.sleep)

Tổng kết nộp bài:

- Chụp màn hình TestNG HTML Report: tổng số test, số pass, số fail, thời gian chạy
- Chụp màn hình cấu trúc thư mục project đúng theo mẫu (dùng IDE file explorer)
- Viết 5 dòng nhận xét ngắn: Framework POM giúp ích gì khi so với cách viết truyền thống ?
- GV sẽ cho thêm bài 3.0 điểm sau khi làm xong 7 bài tập trong lab 9

❖ Lưu ý quan trọng cho toàn bài lab

1. Mọi @Test phải sử dụng getDriver() từ BaseTest - không tạo WebDriver mới trong test class
2. Mọi driver.findElement() phải nằm trong Page Object - không viết trực tiếp trong test
3. Tất cả dữ liệu test (URL, username, password) phải đọc từ file config hoặc Excel/JSON
4. Không dùng Thread.sleep() – sẽ bị trừ 0.5 điểm mỗi lần phát hiện
5. Nộp file Excel test data cùng với source code

HẾT LAB 9*GOOD LUCK!**Được Học Là Niềm Vui – Làm Bài Tập Là Đam Mê!*